

Seminar Paper

Sliding Squares in Parallel

Jonas Spies

Semester: Wintersemester 2025/26
Date of Submission: February 12th 2026
Advisors: Prof. Dr. Alexander Wolff
Antonio Lauerbach, M. Sc.



Julius-Maximilians-Universität Würzburg
Lehrstuhl für Informatik I
Algorithmen und Komplexität

1 Introduction

Imagine a modular robot that can transform into different shapes to accommodate for different situations. The *sliding squares* problem asks for the minimum number of transformations required to transform one connected configuration of such a robot, formally n grid-aligned square-shaped modules, into another without breaking connectivity. While the problem itself is np-hard [AADK⁺22], algorithms to reach a goal configuration in 2-dimensional space within $\mathcal{O}(n \cdot P)$ steps (transformations) have been known for some time [AAD⁺19]. Being the size of the bounding box's perimeter, P is lower-bounded by $\sqrt{n}$ and upper-bounded by n , giving such algorithms a worst-case runtime of $\mathcal{O}(n^2)$. In pursuit of breaking this quadratic runtime barrier, Akitaya et al. have redefined the problem as *parallel sliding squares* to allow for parallel movement of modules and were able to come up with a linear runtime algorithm as a result [AFK⁺25]. This is especially remarkable because these sequential algorithms as well as the parallelized one are worst-case optimal, implying that parallelization can speedup worst-case instances by a linear factor. This paper aims to explain the core idea behind the parallel algorithm while elaborating on some aspects.

1.1 Preliminaries

We will denote the start configuration as C_1 , the target configuration as C_2 and use C for statements that apply to both C_1 and C_2 .

Definition 1. A transformation denotes the movements of modules in C_1 that are required to reach C_2 . We consider a transformation valid if C_1 and C_2 are connected configurations and if all modules stay connected without causing any collisions before, during or after the transformation. During movement, the backbone, i.e. the set of stationary modules, must not be empty and also stay connected, which we will refer to as the backbone property. Furthermore, each module m_1 may only move one square along the grid, either orthogonally or diagonally along the perimeter of a stationary edge-adjacent module m_2 within one transformation.

Figure 1 shows a few valid transformations while Figure 2 gives examples for invalid transformations. Particularly the case where one module m_1 fills the space of its edge-adjacent neighbor m_2 , which is moving in a direction orthogonal to that of m_1 , as shown in Figure 3a requires extra care. Since m_2 must completely finish its movement before m_1 can move collision-free, this edge case requires two transformations instead of one. Similarly, Figure 3b shows a configuration where two diagonally moving modules would collide along the way, if one does not move them sequentially. These edge cases must be accounted for because simply checking if C_1 and C_2 are connected and collision-free (i.e. no two modules are in the same square) does not suffice for them. As a result, many of the following maneuvers will be estimated as requiring two transformations even though in most cases, one would suffice. Also, the backbone property reflects the restriction a practical application may bring with it. For example, the 'top' part of a robot may fall off the 'bottom' part, if the top is fully separated from the bottom due to all modules in the middle moving.

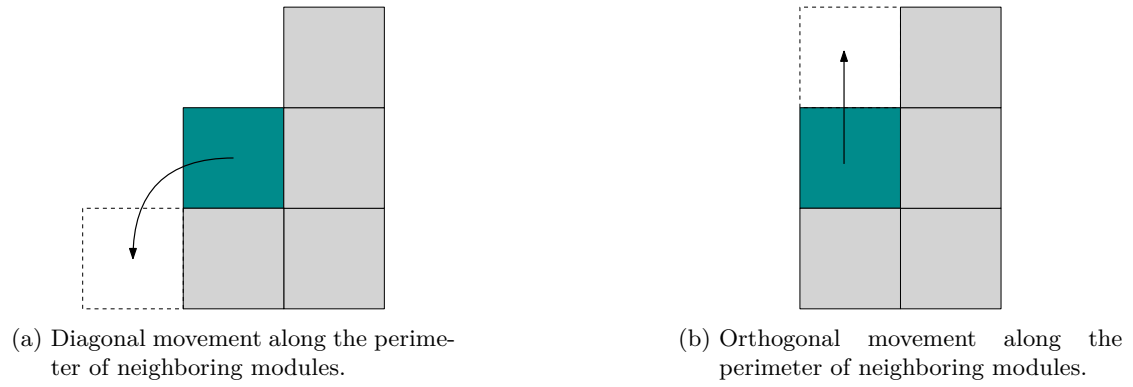


Fig. 1: Two valid examples for a transformation.

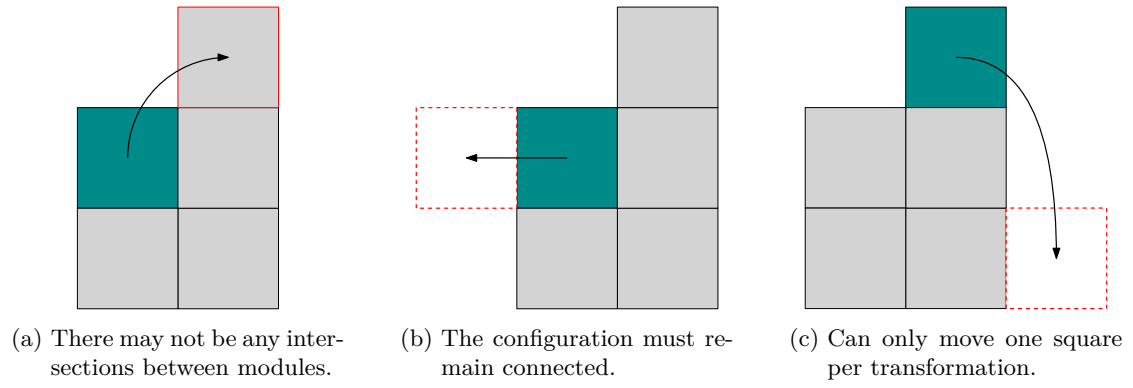


Fig. 2: Invalid examples for a transformation.

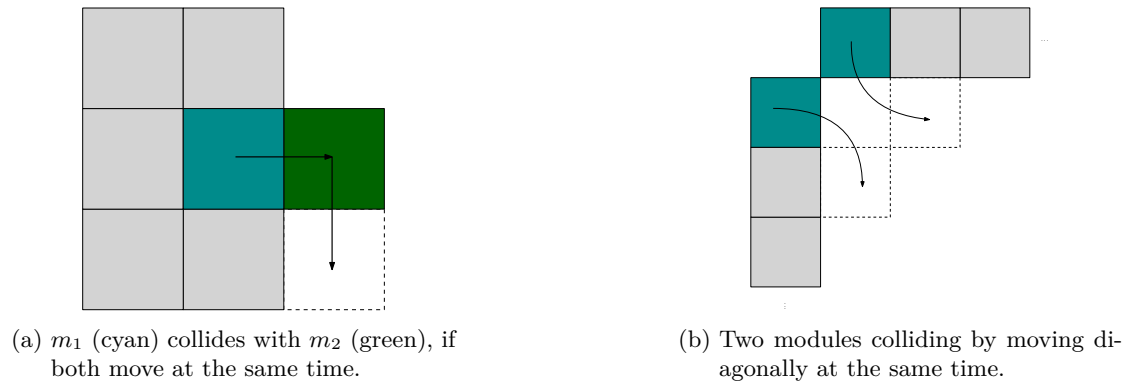


Fig. 3: Two edge cases that require two transformations instead of one.

Definition 2. A schedule from C_1 to C_2 is an ordered list of transformations that, if applied in order, are valid and arrange C_1 to become equivalent to C_2 . The size of a schedule is called its makespan.

We consider the *neighborhood* of a module to consist of the eight spaces surrounding it and a *subconfiguration* of a configuration to be a connected subset of its modules.

2 'Easy' instances

Before solving arbitrary instances, we first consider an easy case where start and end configurations are 3-scaled.

Definition 3. A configuration is scaled by factor k if all of its modules form $k \times k$ squares that are aligned with a $k \times k$ grid.

Figure 4 shows an example 3-scaled configuration. We choose the scaling factor of 3 for its relative compactness and make use of its easy to fulfill backbone property to show the following.

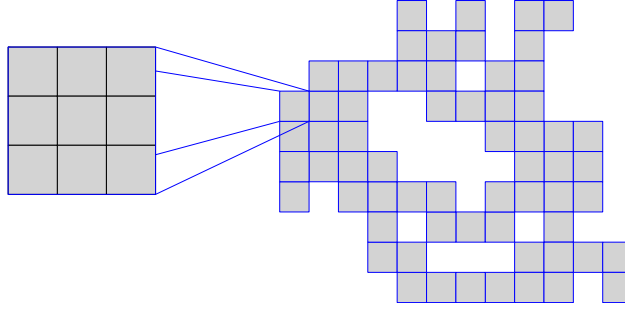


Fig. 4: A 3-scaled configuration. Each module in the configuration (blue outline) corresponds to a 3×3 square of 1-scaled modules (black outline)

Lemma 4. A 3-scaled configuration can be translated orthogonally by one unit within 6 valid transformations.

Proof. A schedule that proves Lemma 4 essentially divides C_1 into multiple 3-wide strips that are aligned with the direction of movement (e.g. vertical strips for moving vertically). Moving all of these strips in parallel, it is clear that adjacent strips will remain adjacent, ensuring that connectivity may only be broken locally. First, we take 3 transformations to move modules from the middle of the strip to the outside, expanding the strip in the target direction. After that, we do another 3 transformations to fill the gaps in the middle with modules at the end of the strip, effectively retracting it. Figure 5 illustrates such a schedule. Connectivity is clearly not broken locally and the backbone stays connected as well: Suppose without loss of generality that we move C_1 downwards. The backbone of each strip would first form a ' $\cap$ ' and then a ' $\cup$ ' shape, ensuring that each strip remains connected to its left and right neighbor at all times. $\square$

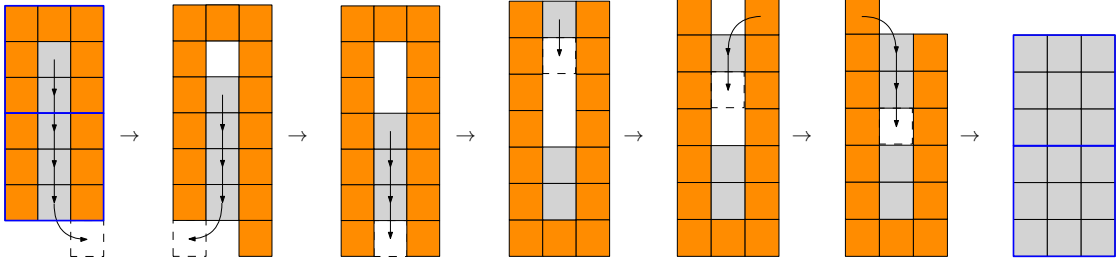


Fig. 5: A 6-step translation by one unit downwards. Modules that form the backbone are highlighted orange.

Corollary 5. *Let $P = 2(a + b)$ be the perimeter of the bounding box of a 3-scaled configuration C with height a and width b . Choosing the coordinate system so that all modules are in the first quadrant and within $P/2$ distance to $(x, y) = (0, 0)$, C can be pushed towards the origin by applying Lemma 4 no more than $(a - 3)$ times downwards and no more than $(b - 3)$ times leftwards to form a configuration C' so that the line between any two modules that share an $x(y)$ -coordinate contains no empty modules. This property is called $x(y)$ -monotonicity.*

By applying corollary 5 to both C_1 and C_2 , we achieve two configurations C'_1, C'_2 that have an xy -monotonous overlap.

Lemma 6. *There exists a schedule of makespan up to $6P$ that transforms two xy -monotonous configurations C'_1, C'_2 into one another.*

Proof. Since C'_1 and C'_2 have some xy -monotonous overlap, we move non-overlapping modules through that overlap along 'L' shapes to create more overlap, gradually transforming one configuration into the other. This can be done within two steps, by first pushing one module in the middle to the right side of the 'L', then filling the gap with a module at the top end of it – refer to Figure 6. Repeating the movement a total of 9 times, once for each 1-scaled module within the 3-scaled module, we get a schedule of size 18. Up to $n - 1$ modules may require moving if the only overlap is in $(0, 0)$, but by moving all modules that lie on a common slope at once, we only need $\mathcal{O}(P)$ steps. We use two such slopes, L_1 containing modules that are exclusively part of C'_2 and L_2 containing modules that are exclusively part of C'_1 . Transforming C'_1 into C'_2 , the modules that are to be moved lie on L_2 while those that are not yet filled, but need to be filled, lie on L_1 . We define these two lines so that no C_2 -exclusive modules lie to the left of L_1 , and no C_1 -exclusive modules lie to the right of L_2 , refer to Figure 7. By computing a maximum matching between exclusive modules on L_1 and those on L_2 , we ensure that in each iteration either L_1 moves right or L_2 moves left, so that P iterations, 18 transformations each, suffice, as shown in Figure 7b. While the paper in question talks about this schedule taking up to $18P$ transformations, a better upper bound is $6P$ because the lines are 3-scaled, so that they move three units at a time. $\square$

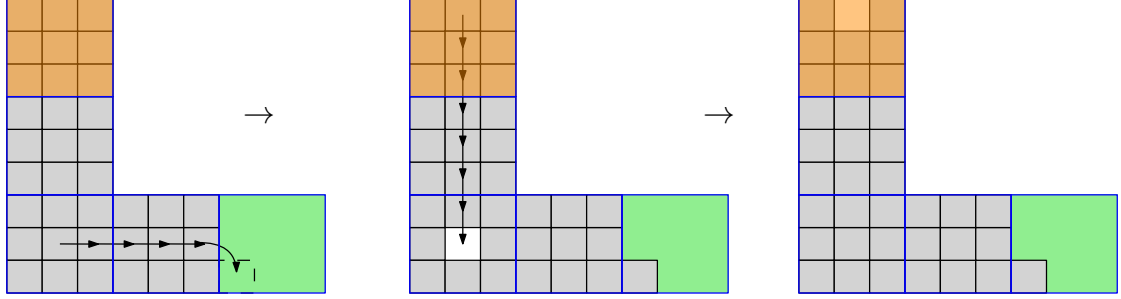
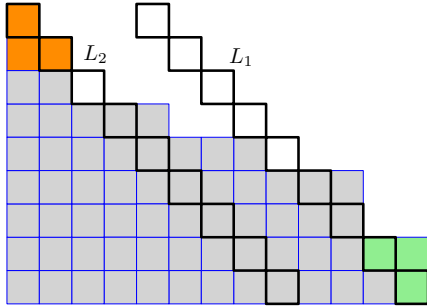
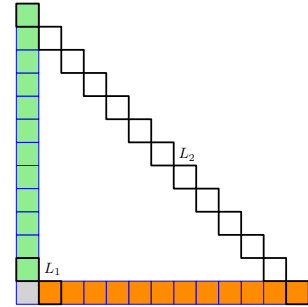


Fig. 6: C'_1 is transformed to move an exclusive module (orange) into the position of a C'_2 -exclusive module (green) within two steps.



(a) An example for how to place the lines L_1, L_2 .



(b) An extreme case that requires $\mathcal{O}(P)$ iterations.

Fig. 7: Grey modules are part of the overlap. Orange modules are exclusively part of C'_1 , while green modules are only in C'_2 . The lines L_1, L_2 are placed according to their definition.

Theorem 7. *By first using Corollary 5 to turn C_1 and C_2 into two xy -monotone configurations C'_1 , C_2 within $6P$ steps, then applying another $6P$ transformations according to Lemma 6, it is possible to solve parallel sliding squares for 3-scaled instances within linear makespan.*

3 Transforming 1-scaled instances to 3-scaled ones

By using a sweep-line algorithm, it is possible to transform an arbitrary configuration into a 3-scaled one, which we will do to both C_1 and C_2 . From there, we can apply Theorem 7 to solve the problem. In order to build a sweep line, however, we first need an exoskeleton, which will serve as a flexible intermediate configuration. That will be constructed by determining a tree-like structure, a skeleton, within C_1 and C_2 and moving modules from the leaves up towards the root to make its neighborhood full.

3.1 Building a skeleton

Since we later need the skeleton to be similar to a tree, we get the following definition:

Definition 8. *A skeleton S is a connected subconfiguration of a configuration C , so that every module $m \in C$ is either in S , or edge-adjacent to a module $m' \in S$. The dual graph of S , where every module is a vertex and there is an edge between two vertices u, v if their respective modules are edge-adjacent in S , must not contain any cycles larger than 4, and no two cycles may intersect.*

The algorithm to obtain a skeleton is fairly straightforward: First, determine the x -coordinate of each module (determined by its bottom-left corner) and add all modules with even x -coordinate to S . After that, we add all remaining modules m that have no left or right neighbor – by construction, m 's neighbors would be in S . That property ensures that in most cases, large cycles can be broken simply by removing some module m on the cycle that has no south neighbor and no north neighbor m_N with $m_N \in S$. To connect these modules, we sweep from left to right using a 3-wide strip. Considering only modules on the strip, we use the dual graph of C to compute the shortest path between any two modules in S , adding edges with corresponding weights between their respective vertices. After that, we compute minimum spanning trees, adding modules to S if they are on a path that corresponds to an edge of the spanning tree. This will take more than linear time, but no modules are moved in the process. Moving the strip by two modules to the right in each iteration, we can compute a skeleton-like structure in polynomial time, that may contain large and intersecting cycles. Having already explained the rough idea of how to break these cycles, the rest comes down to a case analysis.

Next, we can construct a tree-graph G from S in a fashion similar to that of its dual graph. Just like in the dual graph, G consists of a vertex for each module in S , and two modules have an edge in G if they are edge-adjacent in the configuration. The difference is that we will treat 4-cycles as just one vertex. This ensures that G is a tree and won't

form a degree 5 vertex because just like a single module, a 4-cycle can also only have four neighbors, alas it would be a 6-cycle and S would not be a skeleton. Additionally, we construct a second tree G' that is just like G , except that we add vertices for all remaining modules $m \in C$ and arbitrarily connect each of them with an edge to the vertex of one edge-adjacent skeleton module $m' \in S$. We will now show how to turn a subtree G'_r of G' with root r into an exoskeleton.

3.2 Small exoskeletons

Definition 9. An exoskeleton X_r with root r consists of three parts. The core K_r is a modified subtree of G_r . Next, we arbitrarily choose a fixed $j \in \{0, 1, 2, 3\}$ so that every fourth module may be empty (no module corresponds to the vertex in K_r), if its distance to the root, divided by 4, has a remainder of j . The shell consists of vertices in G'_r and nodes in G_r that are themselves leaves or close to a leaf. It encases the core so that all modules corresponding to non-leaf vertices in K_r are surrounded by eight modules (seven if one neighbor is in the core and empty). Any remaining vertices are part of the tail and, by definition, correspond to modules in the neighborhood of leaf nodes in K_r .

An example for an exoskeleton can be found in Figure 8.

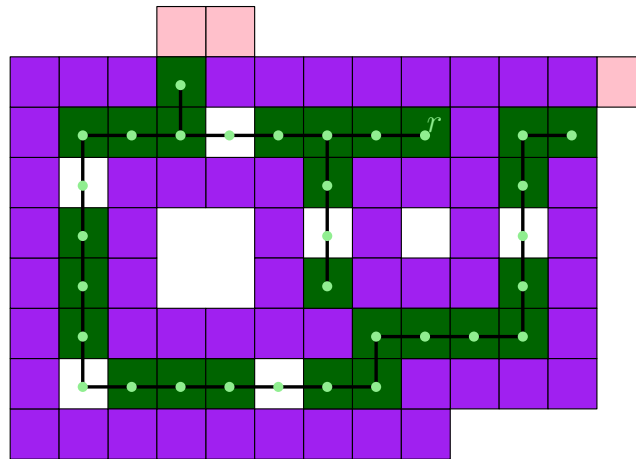


Fig. 8: An example for an exoskeleton. Green modules belong to the core K_r (its tree structure is noted with light green vertices and edges). Shell modules are marked purple, tail modules pink. In this example, j equals 0.

We will show how to turn the leaves of G'_r into exoskeletons by looking at subtrees G'_c that consist of 9 or less vertices: Consider another subtree G'_s of G'_c with root s where each module in it is in the neighborhood of s . If c and s are distinct, the children of s can walk along the perimeter of G_c to enter the neighborhood of c in up to 11 steps. An interesting case arises if walking strictly along the perimeter of G_c is not possible because there is a module o from another part of G' in the way. The module o cannot be part of S , alas S would contain a cycle and not be a skeleton (remember, all modules in G_c are skeleton modules). Thus, we can simply have o walk towards c

and have the module we originally intended to move take its place both geometrically and in the corresponding data structures – Figure 9 illustrates. Since G'_c consists of no more than 9 modules, repeating this process a total of seven times guarantees¹ a 3×3 block within 77 transformations, and such a block is, by definition, an exoskeleton. Realistically, most modules can even be moved in parallel, significantly reducing the number of transformations required in actuality. This yields the following lemma.

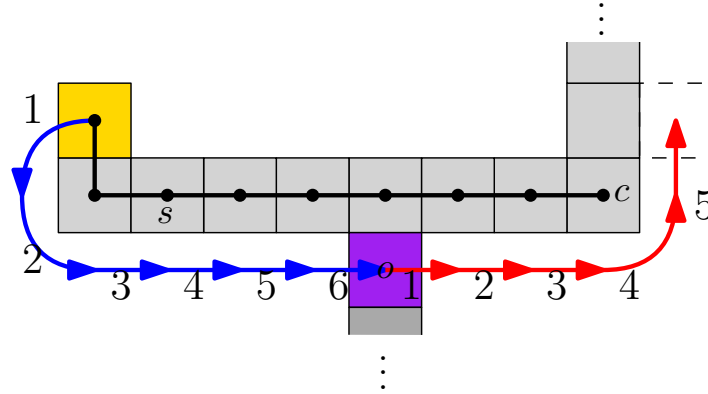


Fig. 9: An example of a subtree G'_c . Skeleton modules are grey. The leaf of G'_s (yellow) moves along the blue arrows and the non-local module o (purple) moves along the red arrows. Without o , this schedule would have a makespan of 11, which would be the worst case possible.

Lemma 10. *A tree of size 9 or less can be transformed into an exoskeleton in a constant number of transformations.*

3.3 Merging to large exoskeletons

After transforming all leaves into small exoskeletons, it is time to walk up the tree, turning other vertices into exoskeleton nodes as well. Consider a node $d \in G_r$ that is not yet part of an exoskeleton, but its children are. If d has no children, it is a leaf and would be part of a small exoskeleton. Therefore, by definition, d has between one and three children c_1, c_2, c_3 . Case analysis shows us that there are at most two empty spaces in the neighborhood of d that, if filled, will turn d into an exoskeleton. Figure 10 gives an example where this is the case. To fill them, we implement two maneuvers which Akitaya et al. call *Inchworm Push* and *Inchworm Pull* that, if applied together, will fill one gap at a time.

Inchworm Push Let e be an empty space in the neighborhood of d . Within two steps, it is possible to fill e with a shell module next to c_1, c_2 or c_3 and push that child into

¹If G'_c does not have enough modules to form a 3×3 block, we form the block around the parent of c instead.

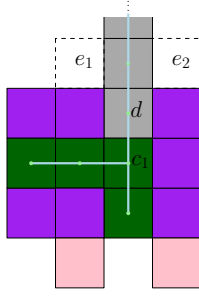


Fig. 10: If d has just one child c_1 that is part of an exoskeleton, there will be two empty spaces e_1 and e_2 in its neighborhood. Recall that shell modules are marked purple, tail modules pink.

the shell fill the resulting gap. The full neighborhood of c_1, c_2, c_3 guarantee that this movement will not break the backbone property. One of the children, denoted c , now corresponds to an empty module, which is a problem if its distance to r has the wrong modularity. To address this, we follow up with Inchworm Pull.

Inchworm Pull In order to fill c again, we go down to one leaf ℓ of K_d , then move all modules corresponding to vertices along the path from d to ℓ up a layer in the tree within 3 transformations. While the paper states this only takes 2 transformations, they forget to consider the case where the core has a 90° -bend with a non-empty module in the middle. As shown in Figure 11, this case would take 3 transformations instead, and more if there are two such 90° -bends without an empty module in-between. While a gap every four modules does prevent two bends without any gaps in a row, we unfortunately cannot guarantee the corner of every bend to be empty, bringing the worst-case number of steps required for Inchworm Pull to 3. Assuming ℓ has a tail or three exclusive

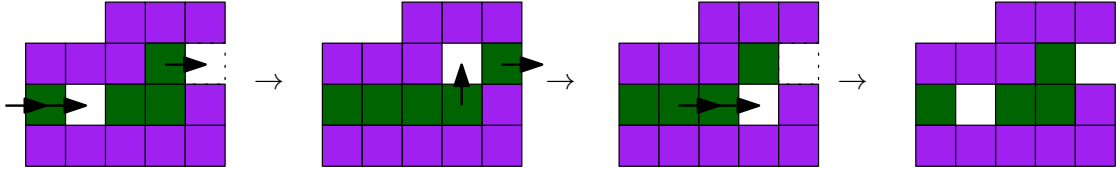


Fig. 11: This part of the core needs 3 transformations to restore its original structure, sending one module towards the root and pulling one from a leaf in the process.

shell modules, one module from there will simultaneously take ℓ 's place and turn any remaining shell modules exclusive to ℓ into a tail. The shell ensures that the backbone property is preserved during movement. To allow for parallel execution of Inchworm Pull, Akitaya et. al. split the maneuver into four phases, three transformations each, but we will not consider this during runtime calculations for a better worst-case estimate.

With a way to build large exoskeletons from small ones, it remains to argue why this can be done in $\mathcal{O}(P)$ transformations. We have already shown that Inchworm Push requires two steps and that Inchworm Pull requires three. No module needs more than

two Inchworm Push / Pull maneuvers, resulting in up to 10 transformations per module. Since we only operate on modules with a corresponding vertex in G'_r , we use the property that no module in G' can have more than three children to find an r so that G_r has $\Theta(P)$ vertices.

4 Building the sweepline

With the exoskeleton in place, the next step is building a sweep line to the right of the configuration's bounding box.

Lemma 11. *An exoskeleton X_r can cross the boundary of the bounding box of C in any direction within $\mathcal{O}(P)$ transformations.*

Proof. We deviate from the paper by not getting rid of empty modules in K_r because this may cause each following Inchworm Pull to require linear instead of constant time. Assume, without loss of generality, that we want to expand X_r to the right. We arbitrarily choose a module s , which is part of the shell and the right neighbor of a right-most module in K_r . By making s the new root of our tree X_r , then using Inchworm Push and Pull three times to make its neighborhood full, we turn s into an exoskeleton module. Repeating this procedure no more than P times concludes the proof. $\square$

We build the sweep line by using Lemma 11 to first expand past the bounding box B by three modules to its right, then, starting at the 3×3 block outside of B , expanding to the top and bottom. It is important that the height of the sweepline is divisible by 3, so we additionally expand one module past B to the top or bottom, if necessary. It remains to argue that Lemma 11 preserves connectivity. While Inchworm Push and Pull are generally safe, the root r of an exoskeleton is assumed to not move during these operations. Since, by construction, r and its shell is the only guaranteed path to the rest of S , and by extension the rest of C , moving it could break connectivity. This could be a problem because we change the root to another module s , so r could become an empty module if, at some point, it is a leaf and Inchworm Pull pulls this leaf. The paper addresses this problem by simply having Inchworm Pull pull other leaves before r instead. However, they did not mention the case where r is the only leaf, which can be the case if X_s is simply a path from s to r with no branches along the way. Figure 12 illustrates.

We suggest the following solution to this problem: By construction, the rest of the configuration is connected by itself and there is a path to r that only involves modules outside of X_r , except for r and one of its shell modules r' . If we keep one side of the shell intact, this side combined with the core suffices as backbone for the other side of the shell modules to move freely. Once r has no more tail modules besides r' (at this point r' may still be shell), we remove r and r' from the exoskeleton so that its parent becomes a new leaf. From there, we can push the new tail module into a gap in the core (at the same time we push a module into the neighborhood of s) and then perform an Inchworm Pull as usual (this one might take 4 transformations since one gap has been filled). Still, in a worst case, the exoskeleton X_s starts as a path from s to r with no

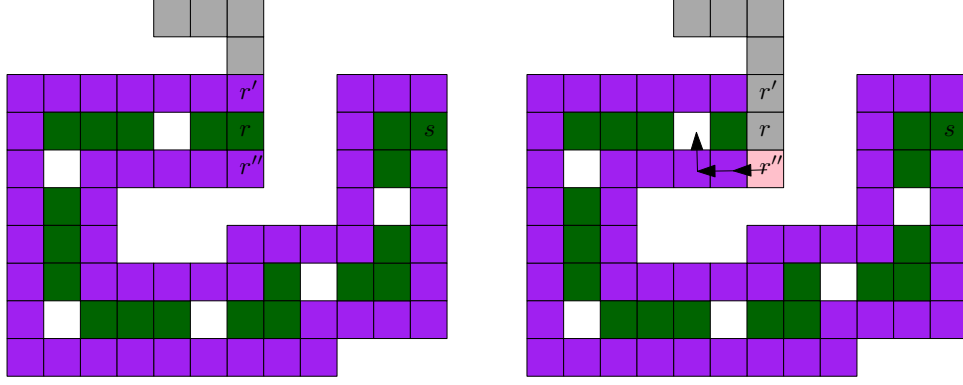


Fig. 12: Left: pushing three more modules into the neighborhood of s would pull r , r' and r'' , cutting connectivity to the rest of C (grey modules). Right: our suggested fix to the problem

branches and there are $P/4$ modules to expand to the right (3 high) and $P/4$ modules² to expand to the top and bottom (3 wide). Since each core module needs a shell, this implies that – assuming no shell module is counted twice (refer to Figure 13) – the shell must contain at least $3 \cdot P/2 + 6$ modules. Therefore, with approximately one third of the modules belonging to the core, the minimum size an exoskeleton should have is $4 \cdot P/2 + 8 \approx 3P$. While the paper recommends an exoskeleton with $36P$ modules if possible, this brings our recommendation down to just $3P$.

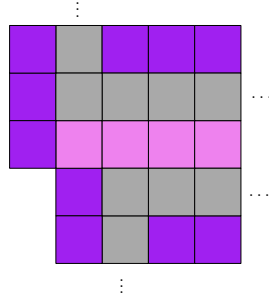


Fig. 13: An example where pink shell modules are counted twice in the sense that it they are the shell of two components of the core.

5 How to sweep

Having built the sweepline, the sweep itself is fairly straightforward: We stop distinguishing between modules as part or not part of an exoskeleton, instead considering the sweepline as a vertical tower of 3×3 blocks, so-called *meta-modules*, that may have an

² P counts height and width twice each, so $P/2$ modules must be traversed in total. While the width may be close to $P/2$, that would imply that the height is close to 1 and vice versa.

empty module in the middle. Consider the top-most meta-module as M_1 and, if there are a total of m meta-modules in total, M_m as the bottom-most one. We introduce a clean operation to make the middle of a meta module empty without pushing any modules to the right, and an advance operation to move the meta-module by one unit to the left. By splitting each M_i into one of two groups depending on if i is an even or odd number, we allow for parallel motion of each group without losing connectivity.

Clean operation Although more intricately discussed in the paper, this essentially comes down to the following case analysis. Consider a meta-module M_i with a module

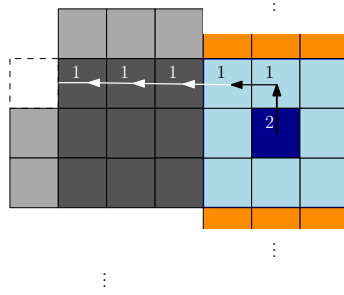


Fig. 14: The clean operation is completed once the middle (dark blue) module is empty. This is achieved by first following the arrows labeled 1, then the other one labeled 2.

in the middle. We want to push it to one of the three spaces to the left of M_i . If these three spaces are occupied, there is a 3 units tall rectangle of width j directly to the left of M_i . If there is any open space above this rectangle and $i \neq 1$, we push the module to the right-most such space, otherwise we simply push it to an empty space left of the rectangle. Since we move in two phases, M_{i-1} does not move and thus no collisions are caused by this. Refer to Figure 14 for one example clean operation, but all cases can be resolved within 2 transformations. Eventually, this may cause the bounding box to be exceeded by more than one unit, but we simply stop cleaning M_i if that case arises – options how to continue from there will be discussed shortly.

Advance operation While this is also simple case analysis, there are some cases that are worth talking about. Although the meta-module M_i advances, the modules it consists of may change in the process. Consider the case where there are three modules to the left of M_i . We denote c as the middle module of M_i , which must be empty before advancing. By moving the left neighbor c' of c to the right, c' becomes the new center of M_i , and there will be three modules to the right of M_i . Other cases may produce up to two modules to the right of M_i , but with a full middle. Regardless, there are schedules of makespan no larger than 6 for each of these cases. In order to reach the initial state (each meta module is empty and no modules are to the right), it suffices to repeat cleaning each module, and filling the middle with a module from the right again. Once the sweepline stops cleaning due to the bounding box being exceeded, there are two options for the meta module: Although the paper continues advancing forward (this

is the case where there are three modules to the left of M_i) without cleaning, simply not advancing M_i anymore works as well and simply changes the position of the empty module to be filled later.

After the sweep Once every meta-module stops advancing, we are close to a 3-scaled configuration. We consider a 3×3 grid over the configuration and particularly, the right-most such 3×3 space at the height of M_1 that contains modules. We let these modules 'trickle' down in 'L' shaped movements to first fill the gaps of $M_1, \dots, M_m$, then to fill the 3×3 spaces at the bottom that contain between one and eight modules. Once the right-most space at the height of M_i contains no more modules, we move down to that of M_{i+1} and continue redistributing the modules until we get a 3-scaled configuration. While there may be up to eight leftover modules, they generally do not pose a problem as long as we bring them to the bottom right of the configuration within $\mathcal{O}(P)$ transformations each.

6 Conclusion

The parallelized algorithm achieves a runtime of $\mathcal{O}(P)$, which is remarkable considering its growth is not linear in the number of modules, but in the size of the configuration's bounding box. This means that, in optimal cases where $P \cong \sqrt{n}$, we actually get a runtime of $\Omega(n^{1/2})$, which is much faster than sequential algorithms that thus far scale with both number of modules and bounding box size for a best-case runtime of $\Omega(n^{3/2})$.

The paper advertises the algorithm as a universal solution to the parallel sliding squares problem, but there are a few edge cases where this is not the case. Particularly building the sweep line may cause problems if there are not branches in the core of the exoskeleton to take modules from rather than the root. While we have suggested a solution that involves taking modules from the shell, it requires more than $2P + 6$ modules in the configuration. As it currently is, strongly elongated starting and end configurations cause undefined behavior for this algorithm. This implies that width and height combined must be no larger than $(n/4) - 2$, so that we get $P = 2(a + b) = n/2 - 4$ and $n = 2P + 6 = 2(n/2 - 4) + 6$ does not produce a contradiction. Let us further narrow down the maximum width and height we can manage with this: All modules must be inside the bounding box, thus $ab \geq n$. Referring to height as a , assume, without loss of generality, that width b is minimal with $b = \frac{n}{a}$, so that the maximal height possible is

the following:

$$a + \frac{n}{a} = \frac{n}{4} - 2 \quad (1)$$

$$\Leftrightarrow a^2 + n = (\frac{n}{4} - 2)a \quad (2)$$

$$\Leftrightarrow -a^2(\frac{n}{4} - 2)a - n = 0 \quad (3)$$

$$\Leftrightarrow a = \frac{-(n/4 - 2) \pm \sqrt{(n/4 - 2)^2 + 4n}}{-2} \quad (4)$$

$$\Leftrightarrow a = \frac{\sqrt{n^2 + 48n + 64} + n - 8}{8} \quad (5)$$

This corresponds approximately to a maximum height of $\frac{n}{4}$ and a minimum width of 4, which is also the limit imposed by a simpler restriction: A line that is n modules long has too few modules to build a 3 wide sweepline of height n . The line would require $3n$ modules instead, and up to six more if the height is not divisible by 3, so that the minimum width of the line is 4.

Furthermore, while technically linear in runtime, the algorithm heavily benefits from the O -notation as there is a big constant multiplier, that we are going to investigate. For 3-scaled instances, we need to push both C_1 and C_2 up to $P/2$ times into the bottom-left corner within a total of $6P$ transformations according to Lemma 4. After that, finding a schedule from C'_1 to C'_2 as described in Lemma 6 takes another $6P$ transformations, solving this problem in a total of $12P$ transformations. The 1-scaled variant additionally requires the construction of an exoskeleton of size $3P$ as suggested by us. The default case involves up to two instances of Inchworm Push and Pull each for a total of 10 transformations, giving an estimate of $30P$ steps to build the exoskeleton. From there, building the sweep line requires pushing and pulling (5 steps each) approximately $3P/2$ modules, assuming we stretch the 3-wide exoskeleton once along the width to reach the bounding box, then build the sweep line that is as high as the bounding box – for a total of $7.5P$ transformations. Lastly, the sweep moves once along the width of the bounding box, which we estimate as $P/4$. With every advance taking up to six steps and requiring up to three cleaning operations, costing 2 transformations each, plus one extra transformation to fill a meta module with a module right to it, we get a total of $30P/4$ steps for the sweep – accounting for both odd and even numbered meta modules. Assuming each of the $P/12$ meta modules³ terminates with an empty middle requires $P/12$ 'L' movements, two steps each. Next, we assume that every meta-module leaves a module that is not yet 3-scaled, with eight modules each in the upper half and one each in the lower half. This would yield $P/4/3/2 = P/24$ 3×3 spaces that each need eight modules to trickle down into in 'L' shapes, for a total of $P/2$ transformations caused by 'L' shaped movements in this stage.

This results in a total of $45.5P$ transformations, which need to be applied to both C_1 and C_2 . Lastly, applying Theorem 7 to this instance then brings the total runtime up to a total of $103 \cdot P$ transformations. In a different paper, Akitaya et. al. tested a

³Assuming $P/4$ height and every meta module is 3-scaled

sequential $\mathcal{O}(Pn)$ algorithm called Gather&Compact and were able to solve an instance with $P = 400$ and $n = 5000$ (50% density) within an average of 150,666 steps [ADK⁺21]. For a higher density of 85%, resulting in $n = 8500$, Gather&Compact was even faster, only taking 69,488 steps despite expectations. According to our estimate, the parallel algorithm would solve both instances in 41,200 transformations, implying that in practice, while still better, the algorithm may perform worse than one might expect in comparison. This could be the case particularly on small instances, e.g. $P = 40, n = 50$, where Gather&Compact needed 237 transformations, but our estimates for the parallel algorithm add up to 4120. That implies that our algorithm might only be faster for large instances. Additionally, there seems to be an effect where sequential algorithms perform well on dense instances, which could further diminish the effectiveness of our algorithm considering it only works on somewhat dense configurations, i.e. $P \leq n/2 - 4$.

References

- [AAD⁺19] Hugo A. Akitaya, Esther M. Arkin, Mirela Damian, Erik D. Demaine, Vida Dujmović, Robin Flatland, Matias Korman, Belén Palop, Irene Parada, André van Renssen, and Vera Sacristán: Universal reconfiguration of facet-connected modular robots by pivots: The $o(1)$ musketeers, 2019. <https://arxiv.org/abs/1908.07880>.
- [AADK⁺22] Hugo A. Akitaya, Erik D. Demaine, Matias Korman, Irina Kostitsyna, Irene Parada, Willem Sonke, Bettina Speckmann, Ryuhei Uehara, and Jules Wolms: Compacting Squares: Input-Sensitive In-Place Reconfiguration of Sliding Squares. In Artur Czumaj and Qin Xin (editors): *18th Scandinavian Symposium and Workshops on Algorithm Theory (SWAT 2022)*, volume 227 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 4:1–4:19, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, ISBN 978-3-95977-236-5, 10.4230/LIPIcs.SWAT.2022.4.
- [ADK⁺21] Hugo A. Akitaya, Erik D. Demaine, Matias Korman, Irina Kostitsyna, Irene Parada, Willem Sonke, Bettina Speckmann, Ryuhei Uehara, and Jules Wolms: Compacting squares: Input-sensitive in-place reconfiguration of sliding squares, 2021. <https://arxiv.org/abs/2105.07997>.
- [AFK⁺25] Hugo A. Akitaya, Sándor P. Fekete, Peter Kramer, Saba Molaei, Christian Rieck, Frederick Stock, and Tobias Wallner: Sliding squares in parallel, 2025. <https://arxiv.org/abs/2412.05523>.